CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

Project Report
ON
Student Record Management System

DATA STRUCTURES LAB
SUBJECT CODE – 25CAP-152

SUBMITED BY:

Name: Subham Thakur UID: 25BCD10016

Branch: UIC Group: A, Section: BCD-1

Semester: 2

SUBMITED TO:

Name: Riya Mondal Ma’am

Designation: AP

Sign: ----------

INDEX

1. Introduction about the Topic

TheStudent Record Management System is aconsole-basedapplication developed in the C programming language. It is designed to manage student academic records efficiently using fundamental data structure concepts such as arrays and structures.

The system allows users to add new students, view all existing records, update student information, and delete records. Beyond basic CRUD operations, the application implements three classic sorting algorithms — Bubble Sort, Selection Sort, and Quick Sort — to organize records by marks. It also supports Binary Search for fast retrieval by marks and Linear Search by student name.

This project is a practical application of Data Structures concepts taught in the course, demonstrating how real-world problems can be solved using arrays, structures, and algorithm design in C.

2. Objectives

Thekeyobjectivesofthis project are:

- To implement a Student Record Management System using arrays and structures in C.
- To provide complete CRUD functionality: Add, Display, Update, and Delete student records.
- To implement and demonstrate Bubble Sort, Selection Sort, and Quick Sort algorithms for sorting by marks.
- To implement Binary Search for searching students by marks and Linear Search by name.
- To automatically assign grades (A+, A, B, C, D, F) based on marks.
- To prevent duplicate student IDs using validation logic.
- To build a menu-driven, user-friendly console interface.
- To strengthen understanding of pointers, arrays, structures, and algorithm complexity in C.

3. Data Description

Each student record is stored as a structure (struct Student) with the following fields:

Field	Data Type	Description
id	`int`	Unique student identifier
name	`char[50]`	Full name of the student
age	`int`	Age of the student
marks	`float`	Marks obtained out of 100
grade	`char[3]`	Auto-assigned grade: A+, A, B, C, D, or F

All student records are stored in a global array: struct Student students[MAX], where MAX = 100. A global integer 'count' tracks the current number of records.

4. Tools and Technologies

Technology / Tool	Purpose
C Language (C99)	Primary programming language used for all logic and data structures
GCC Compiler	Used to compile the C source code from the terminal
VS Code	Source code editor used for writing and editing the program
Terminal / CMD	Used to run, test, and interact with the console application
GitHub	Version control and repository hosting for the project
Standard Libraries	stdio.h, stdlib.h, string.h — for I/O, memory, and string operations

4.1 Code with Explanation

The project is written as a single C file. Below is a breakdown of each component:

A) Structure Definition

The struct Student holds all data for one student. An array of size 100 stores all records, and a global counter tracks how many records exist.

```
struct Student {
	int   id;
	char  name[50];
	int   age;
	float marks;
	char  grade[3];
};
struct Student students[100];
int count = 0;
```

B) addStudent() — Add a New Record

Reads input from the user, checks for duplicate IDs, auto-assigns the grade, and pushes the new record into the array.

```
void addStudent() {
	struct Student s;
	printf("Enter ID: "); scanf("%d", &s.id);
	// Check for duplicate ID
	for (int i = 0; i < count; i++)
	if (students[i].id == s.id) { printf("ID exists!"); return; }
	printf("Enter Name: "); scanf(" %[^\n]", s.name);
	printf("Enter Marks: "); scanf("%f", &s.marks);
	assignGrade(&s);   // auto-assign A+/A/B/C/D/F
	students[count++] = s;
}
```

C) Bubble Sort — Sort by Marks (Descending)

Compares adjacent elements and swaps if the left is smaller. Repeats for n-1 passes. Time Complexity: O(n²).

```
void bubbleSort()
                  {
      struct Student temp;
      for (int i = 0; i < count-1; i++)
      for (int j = 0; j < count-i-1; j++)
            if (students[j].marks < students[j+1].marks) {
            temp = students[j];
            students[j] = students[j+1];
            students[j+1] = temp;
            }
}
```

D) Selection Sort — Sort by Marks (Descending)

Finds the maximum element in the unsorted portion and places it at the current position. Time Complexity: O(n²).

```
void selectionSort()
                    {
      int maxIdx;
      for (int i = 0; i < count-1; i++) {
      maxIdx = i;
      for (int j = i+1; j < count; j++)
            if (students[j].marks > students[maxIdx].marks) maxIdx = j;
      // Swap students[i] and students[maxIdx]
      struct Student temp = students[i];
      students[i] = students[maxIdx];
```

```
      students[maxIdx] = temp;

      }

}
```

E) Quick Sort — Sort by Marks (Descending)

Uses divide-and-conquer. A pivot is chosen; elements larger than the pivot go left, smaller go right. Recursively sorts both halves. Time Complexity: O(n log n) average.

```
int partition(int low, int high) {

      float pivot = students[high].marks;

      int i = low - 1;

      for (int j = low; j < high; j++)

      if (students[j].marks >= pivot) {

            i++;

            // swap students[i] and students[j]

      }

      // swap students[i+1] and students[high]

      return i + 1;

}

void quickSort(int low, int high) {

      if (low < high) {

      int pi = partition(low, high);

      quickSort(low, pi-1);

      quickSort(pi+1, high);

      }

}
```

F) Binary Search — Search by Marks

Sorts the array first, then applies Binary Search. Compares mid element with target and halves the search space each step. Time Complexity: O(log n).

```
int low = 0, high = count - 1;


while (low <= high) {
      int mid = (low + high) / 2;
      if (students[mid].marks == target) { found = mid; break; }
      else if (students[mid].marks < target) low  = mid + 1;
      else                                   high = mid - 1;
}
```

4.2 Output (Sample Run)

Below is a sample console output when running the Student Record Management System:

Main Menu:

```
  ╔═══════════════════════════════════════╗
  ║   Student Record Management System    ║
  ╚═══════════════════════════════════════╝


 ── Main Menu ──────────────────────
  1. Add Student
  2. Display All Students
  3. Delete Student
```

```
4. Update Student

5. Sort Records

6. Search Record

0. Exit

————————————————————————

Enter your choice: 1
```

Adding a Student:

```
Enter Student ID   : 101

Enter Name       : Harpreet Singh

Enter Age        : 20

Enter Marks (0-100): 87.5

[✓] Student added successfully!
```

Display All Students:

```
ID    Name                  Age   Marks    Grade

---   --------------------  ---   ------   -----

101   Harpreet Singh        20    87.50    A

102   Amandeep Kaur         21    92.00    A+

103   Rajveer Sharma        19    74.00    B

104   Simran Bhatia         22    55.50    D
```

After Bubble Sort (Descending by Marks):

```
[✓] Sorted by Marks (Bubble Sort - Descending)

ID    Name                  Age   Marks    Grade

---   --------------------  ---   ------   -----
```

```
102   Amandeep Kaur          21    92.00    A+

101   Harpreet Singh         20    87.50    A

103   Rajveer Sharma         19    74.00    B

104   Simran Bhatia          22    55.50    D
```

Binary Search Result:

```
Enter Marks to search: 74

[✓] Student Found!

ID: 103 | Name: Rajveer Sharma | Marks: 74.00 | Grade: B
```

5. Conclusion

The Student Record Management System successfully demonstrates the practical application of Data Structures concepts in C. The project implements all core features including Add, Display, Update, and Delete operations, along with three sorting algorithms — Bubble Sort, Selection Sort, and Quick Sort — and two search techniques — Binary Search and Linear Search.

The use of arrays and structures in C provides a clear understanding of how data can be organized and manipulated efficiently in memory. The sorting algorithms show different approaches to ordering data, each with its own time complexity trade-offs, while Binary Search demonstrates the efficiency advantage of O(log n) over linear methods.

This project reinforces the importance of algorithm selection in real-world applications and serves as a strong foundation for understanding more advanced data structures such as linked lists, trees, and hash tables.

6. Learning Outcomes

Thefollowinglearningoutcomeswere achievedthrough this project:

- Gained hands-on experience with structures (struct) in C and how to use arrays of structures to manage real-world data.
- Implemented and understood Bubble Sort, Selection Sort, and Quick Sort — comparing their logic and time complexities ($O(n^2)$ vs $O(n \log n)$).
- Implemented Binary Search on a sorted array and understood the importance of sorted data for efficient searching.
- Developed skills in writing modular C code with functions for each operation — improving readability and maintainability.
- Understood input validation techniques in C, such as detecting duplicate IDs and handling invalid menu choices.
- Learned how to build interactive, menu-driven console applications in C using do-while loops and switch-case.
- Practiced using string functions (strcpy, strcasecmp) and I/O functions (scanf with format specifiers) effectively in C.
- Gained experience using Git and GitHub for version control, source code management, and project submission.

7. GitHub Link

Thecomplete sourcecode ofthisproject is hosted on GitHub. The repository includes the full C source file with all features implemented.

GitHub Repository:

Note: Replace the above URL with your actual GitHub repository link before submission.